

Interfaces in Java

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Limitations of inheritance

- With inheritance, all members are inherited
- You cannot selectively inherit members

```
// Car class
class Car {
    // Battery capacity is only applicable
    // to electric cars
    public int batteryCapacity;

    void drive() {

    }
}
```

```
// Toyota class
class Toyota extends Car {
    void drive(){

    }
}

// Man class
public static class Main {

    public static void main(String[] args) {
        Toyota myToyota = new Toyota();
        // All cars will inherit the "batteryCapacity"
        // even if they are not electric cars
        System.out.println(
            myToyota.batteryCapacity);    // Not applicable
    }
}
```

Creating interfaces

- Interfaces provide a way to selectively inherit properties and methods
- Classes use interfaces using the `implements` keyword

```
// ElectricCar Interface
interface ElectricCar {

}
```

```
// Tesla implementing the
// ElectricCar interface
class Tesla implements ElectricCar {

}
```

Adding properties to interfaces

- Properties use `UPPER_SNAKE_CASE` by convention
- Value must be set immediately and **cannot** be changed
- Properties are `public static final` behind the scenes

```
interface ElectricCar {  
    // Implicitly public static final  
    // Cannot be changed  
    int BATTERY_CAPACITY = 310;  
  
}
```

Adding interface methods

- Interfaces are typically used to house methods
- The methods are **abstract** by default without code implementation
- Interfaces can have concrete methods that have code implementation
- All **abstract** methods must be implemented in the subclass

```
interface ElectricCar {  
    // Implicitly public static final  
    // Cannot be changed  
    int BATTERY_CAPACITY = 310;  
  
    // This is an abstract method by default  
    void charge();  
  
    // This is a concrete method  
    void autoPark() {  
  
    }  
}  
  
class Tesla implements ElectricCar {  
    // Must be implemented  
    public void charge() {  
  
    }  
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Introduction to polymorphism

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Demonstrating polymorphism

- Polymorphism is a concept in Object-Oriented Programming (OOP) that allows objects to exist in multiple forms
- Showcasing class inheritance

```
// Abstract Car class
abstract class Car {

    // Abstract drive method
    abstract void drive();
    // No Code Implementation

}
```

```
// Toyota class inherits Car class
class Toyota extends Car {

}
```

```
// Tesla class inherits Car class
class Tesla extends Car {

}
```

```
// Lamborghini class inherits Car class
class Lamborghini extends Car {

}
```


Overriding methods

- The `@Override` keyword allows you to create custom method implementations

```
// Abstract Car class
abstract class Car {

    // Abstract drive method
    abstract void drive();
    // No Code Implementation

}
```

```
class Toyota extends Car {
    @Override // Keyword used to override
    void drive() {
        // Toyota specific implementation code
    }
}
```

```
class Tesla extends Car {
    @Override
    void drive() {
        // Tesla specific implementation code
    }
}
```

```
class Lamborghini extends Car {
    @Override
    void drive() {
        // Lamborghini specific implementation code
    }
}
```

Overriding interface methods

```
// ElectricCar interface
interface ElectricCar {

    abstract void charge();
    // No Code Implementation

}
```

- `@Override` is recommended when using interface methods

```
// Tesla class implementing
// ElectricCar interface
class Tesla implements ElectricCar {

    // charge method must be implemented
    @Override
    void charge(){
        // Tesla specific charge implementation
        // Selective inheritance with interface
    }

}
```

Overloading methods

- Overloading allows methods with the same name, to have different implementations

```
// Toyota class
class Toyota {

    void drive() {
        // First implementation of drive method
    }

    // Overloaded drive method
    void drive(int topSpeed) {
        // Second implementation of drive
    }
}
```

Overloading the constructor

- Constructors can be overloaded to have multiple in the same class

```
// Honda class
class Honda {
    // First Constructor
    public Honda(String color,
                  String model) {

    }

    // Second Constructor
    public Honda(String color,
                  String model,
                  String licensePlate) {

    }
}
```

```
public class Main {
    public static void main(
        String[] args) {

        // First Constructor usage
        Honda hondaOne =
            new Honda("Red", "Accord");

        // Second Constructor usage
        Honda hondaTwo =
            new Honda("Red", "Civic", "FST-1977");
    }
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Wrap-up

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Chapter 1 - object-oriented programming (OOP) Java introduction

- Classes as blueprints
- Cookie cutter & Cookie example
- Methods with `void`, `return` keywords
- Creating objects and instances with `new` keyword

Chapter 2 - encapsulation and inheritance

- Access modifiers: `public` , `private` , `static`
- Encapsulation
- Inheritance: `extends`
- Base class and sub class
- Abstract classes: `abstract`

Chapter 3 - interfaces and polymorphism

- Interfaces `interface` , `implements` , `final`
- Polymorphism
 - `@Override`
 - Overload

Next steps

- Internet Sources
 - [Oracle's Java site](#): the stewards of Java technology (Oracle) web site
 - [dev.java](#): Java developer news, events, and other information
 - [Open JDK](#): community dedicated to a free and open-source implementation of Java

Congratulations!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA